Passive TLS Client and Server Fingerprinting with JA3/JA3S and JA4/JA4S

Vadithya Harsha Vardhan Nayak
Roll No.23110349

Sailaja Mudu
Roll No.23110288

Banoth Arvindh
Roll No.23110058

I. Introduction

More than 90% of web traffic is now encrypted with TLS, so network monitoring can no longer read HTTP headers or payloads. The TLS handshake is different: the *ClientHello* and *ServerHello* messages are sent in clear text before encryption starts (TLS 1.2 and 1.3 [4]). Every TLS library and application fills them differently: which cipher suites, which extensions in which order, which elliptic-curve groups, which ALPN protocols. A compact hash of these fields is a *fingerprint* of the software that opened the connection. JA3 and JA3S (Salesforce, 2017) [1], [2] and their successors JA4 and JA4S (FoxIO, 2023) [3] are the de-facto standards, used by intrusion-detection systems, CDNs and threat-intelligence feeds to detect malware command-and-control (C2), bots and unauthorised software without decrypting anything [5], [6].

We built **tlsfp**, a passive fingerprinting tool written in Python. It captures packets live (Scapy over Npcap/libpcap) or reads pcap files, reassembles TCP streams, parses ClientHello and ServerHello messages, computes JA3, JA3S, JA4 and JA4S, and identifies the client against a curated reference database. Identification is evidence-based: JA3 and JA4 are looked up independently and the verdict states whether they agree, conflict, are ambiguous or unknown. Our main contributions are:

- A passive extractor for all four fingerprints that handles multi-segment hellos, out-of-order segments, IPv4/IPv6 and TLS on any port.
- Validation against published references: 135/136 JA4 and 76/76 JA4S values from FoxIO's own test suite, the published Salesforce examples, and 225/225 JA3 and 202/202 JA3S against Wireshark. The checks also exposed two inconsistencies inside the published references.
- A curated database of 9 locally observed clients (Chrome, Edge, Firefox, curl, PowerShell, Python urllib, Python requests, a custom Python script and a hand-built TLS client) plus 16 published, cited fingerprints including malware.
- A held-out evaluation: 75/75 connections identified with JA4 (+JA3) versus 44/75 with JA3 alone; a measurement of browser randomisation; unknown-client handling; spoofing (10/10 successful); and server-side false positives.

II. Problem Statement

Given only a passive copy of network traffic, determine which client application or TLS library opened each TLS connection, and fingerprint the server's response, without decrypting anything and without cooperation from either endpoint.

Input: packets from a live interface or a pcap/pcapng file. **Output:** for each TLS connection the endpoints, SNI, JA3, JA4 (client), JA3S, JA4S (server), and a verdict: a set of candidate labels, an identification status and the provenance of the matching reference entries. Formally, with reference lookups L4(JA4) and L3(JA3) returning sets of labels, the verdict is their intersection when both are non-empty and overlap, a *conflict* when they are disjoint, and the non-empty set otherwise; a verdict is an identification only if exactly one label remains.

Assumptions and constraints: TLS runs over TCP and the hello is visible to the sensor; reference fingerprints exist for the clients to be identified; everything runs on a commodity Windows laptop with no special hardware. **In scope:** TLS 1.0 to 1.3 over TCP on any port, IPv4 and IPv6, client and server fingerprints, curated and published reference data. **Out of scope:** QUIC/HTTP3 and DTLS, decryption, the other JA4+ methods (JA4H, JA4X, JA4T), and line-rate capture.

Scenario. A security analyst sees an HTTPS request whose User-Agent says Chrome. If its ClientHello carries the fingerprint of Python's OpenSSL, the request comes from a script, not a browser. The same idea flags a malware implant whose hard-coded TLS configuration matches a published C2 fingerprint.

Gap. Wireshark computes JA3 and JA4 but has no reference database, no identification and no JA4S. Public JA3 lists are largely stale, and since Chrome 110 (2023) browsers shuffle their TLS extensions on every connection, which breaks JA3 for browsers [7]. Many tools pick one hash and silently ignore the other. We combine both hashes with explicit evidence, validate every algorithm against independent references, and measure the result on current client versions with ground truth.

III. Project Objectives

O1. Passively extract JA3, JA3S, JA4 and JA4S from live and offline traffic, including hellos split over several TCP segments; target 100% agreement with an independent implementation.

O2. Validate the extractor against published reference values (Salesforce, FoxIO); every difference must be explained.

O3. Curate a reference database with at least 5 distinct clients (browsers, curl, custom scripts), recording the source of every entry.

O4. Identify at least 5 clients on held-out traffic and report accuracy, precision, recall and F1 for JA3 only, JA4 only and the combination.

O5. Quantify fingerprint randomisation: distinct JA3 and JA4 values per client over repeated connections.

O6. Demonstrate unknown-client handling and fingerprint spoofing, and analyse server-side (JA3S/JA4S) fingerprints for security monitoring.

O7. Make every result reproducible from committed captures with a single script.

IV. Architecture and Design

Fig. 1 shows the pipeline. Each stage is a separate module with a narrow interface, so the capture source can be replaced without touching the fingerprint logic.

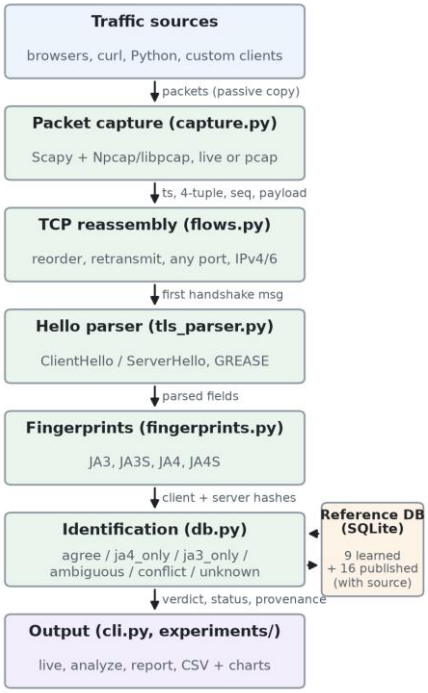


Fig. 1. tlsfp architecture and data flow.

Packet capture `(capture.py)` reads live packets with Scapy over Npcap/libpcap, or pcap/pcapng files, and turns each TCP segment into (timestamp, 4-tuple, sequence number, flags, payload). It strips Ethernet padding and also decodes link types Scapy leaves raw. **TCP reassembly** `(flows.py)` keeps one byte buffer per direction, keyed by the 4-tuple, at offsets relative to the SYN. It accepts out-of-order, duplicate and overlapping segments and handles sequence wrap-around. It stops as soon as the first handshake message is complete, so memory per flow is bounded (64 KiB) and bulk data is never buffered. TLS is recognised by content (record type 0x16, version 3.x, handshake type 1 or 2), not by port. A session table pairs each ServerHello with the ClientHello of the reverse flow. **Hello parser** `(tls_parser.py)` is a dependency-free, bounds-checked decoder of the fields the fingerprints need, including GREASE detection (RFC 8701 [8]) and HelloRetryRequest. **Fingerprint engine** `(fingerprints.py)` implements the four algorithms (Table I). **Reference database and identification** `(db.py)` store fingerprints in SQLite as (kind, value, label, category, source, raw, notes) and every observed connection for historical reports. **Output** is a CLI (live, analyze, evaluate, validate, report) and the experiment scripts.

Table I. Fingerprint definitions as implemented

Name	Input fields (GREASE removed)	Hash
JA3	version, ciphers, extensions (wire order), groups, point formats	MD5
JA3S	version, chosen cipher, extensions	MD5
JA4	a: t, TLS version, SNI d/i, #ciphers, #extensions, ALPN; b: sorted ciphers; c: sorted extensions (no SNI/ALPN) + signature algorithms	a_SHA256(b)[:12]_SHA256(c)[:12]
JA4S	a: t, version, #extensions, ALPN; b: chosen cipher; c: extensions (wire order)	a_b_SHA256(c)[:12]

A. Identification design

JA3 and JA4 are looked up independently. If both match and share a label, the status is `agree`. If only one matches it is `ja4_only` or `ja3_only`. If both match different applications it is `conflict` and both answers are shown. If more than one label remains it is `ambiguous`, and if nothing matches it is `unknown`. Only the first three count as an identification, so the tool never invents a label for an unknown client. A JA3 match can narrow a JA4 that several applications share. Server hellos use the same logic on JA4S/JA3S, but the CLI prints server matches only as hints (Section VII-F).

B. Design decisions and trade-offs

- **Python and Scapy** instead of C, Zeek or a kernel path: portable to Windows, short, and easy to

validate. The cost is throughput (Section VII-G). Because the parser is independent of the capture source, a faster source can be added; an optional Linux XDP source exists but was not evaluated.

- **Own minimal reassembly** instead of relying on tshark: Chrome's post-quantum key share (X25519MLKEM768, group 0x11EC) makes its ClientHello larger than one TCP segment, so per-packet parsing misses it. Stopping after the first message keeps reassembly cheap.
- **Content-based detection** instead of port 443: C2 often uses unusual ports. The cost is inspecting the first payload of every TCP flow.
- **SQLite** instead of a server database: zero setup for a demo laptop. A Redis/Valkey mirror is available for multi-sensor deployments but was not used in the experiments.
- **Exact matching with explicit statuses** instead of fuzzy or ML matching: exact matches are explainable and are how JA3/JA4 are used in practice; fuzzy matching could absorb version drift but would raise false positives.

V. Implementation

The tool is about 2,500 lines of Python 3.13 in the `tlsfp` package plus scripts. Libraries: Scapy 2.8 (capture and pcap I/O), psutil (mapping local ports to processes for live ground truth), the standard `sqlite3`, `hashlib` and `struct` modules, matplotlib (charts) and pytest (34 tests). Hardware is an ordinary laptop with its Wi-Fi adapter (Section VI). The two core algorithms are:

```
feed(segment):            # one per direction
  if SYN: base = seq+1; return
  if new stream and payload does not start
     with 16 03 xx .. 01|02: return  # not TLS
  off = (seq - base) mod 2^32
  store payload at off (keep longest)
  data = contiguous bytes from offset 0
  msg = first complete handshake message
  if msg: stream done; return parse(msg)

identify(ja4, ja3):
  L4 = labels(ja4); L3 = labels(ja3)
  if L4 and L3:
     S = L4 & L3
     if not S: return CONFLICT(L4, L3)
  else: S = L4 or L3
  if not S: return UNKNOWN
  if |S| > 1: return AMBIGUOUS(S)
  return S  # agree | ja4_only | ja3_only
```

A ClientHello builder (`builder.py`) writes raw hellos from a specification, a JA3 string or a JA4_r string. It powers the unit tests, the hand-crafted client and the spoofing experiment. `gen_traffic.py` runs each client in its own process, one at a time, and writes a manifest with each run's label and time window; `experiments/run_experiments.py` regenerates every table and chart in this report.

A. Challenges and solutions

- **Multi-segment hellos.** Chrome's ClientHello was split over two segments (1412 + 574 bytes); solved by the reassembler above.
- **GREASE in signature algorithms.** Chrome and Edge put a GREASE value inside signature_algorithms. The JA4 specification ignores GREASE everywhere and FoxIO's expected output for its sigalg-grease capture confirms this, but Wireshark 4.6.7 keeps it, so its JA4 for Chrome changes every connection. We follow the specification; our validator reports this case separately.
- **Conflicting evidence.** The first version preferred JA4 silently when JA3 disagreed. The status design above replaced it.
- **Ground truth for browsers.** Browsers ran headless with a fresh profile and QUIC disabled, otherwise they use HTTP/3 over UDP, which the sensor does not see. Firefox needed its own profile and the preference network.http.http3.enable=false.
- **Link types.** FoxIO's IPv6 test capture uses BSD loopback framing (address family 30), which Scapy left undecoded and our tool missed. We added a fallback decoder; this bug was found only through the published test captures.
- **Privacy.** Raw captures contained the laptop's unrelated background browsing. A filter keeps only the experiment's connections before captures are shared.

Code repository:
https://github.com/HARSHA219219/Passive-TLS-Fingerprinting-with-JA3-JA3S-and-JA4-JA4S.git

VI. Testbed Setup

All experiments ran on one laptop that was both the traffic source and the passive sensor: the tool sniffed the laptop's own Wi-Fi interface while the clients connected over the Internet to `www.wikipedia.org` (HTTPS, port 443). Table II lists the environment.

Table II. Hardware, software and configuration

Item	Value
Laptop	HP Pavilion 15-eg3xxx, Intel Core i7-1360P (12 cores / 16 threads), 15.6 GB RAM
Network	Realtek RTL8852BE Wi-Fi 6 adapter (interface "Wi-Fi 3"), Internet path to www.wikipedia.org
OS / capture	Windows 11 Home 10.0.26100; Npcap 1.78; Wireshark/tshark 4.6.7
Tool	Python 3.13.5 (OpenSSL 3.0.16), Scapy 2.8.0, psutil 7.2.2, matplotlib 3.11.2
Clients	Chrome 156.0.8078.81, Edge 156.0.4314.58, Firefox 144.0.2, curl 8.21.0 (Schannel), Windows PowerShell 5.1, Python urllib and requests, custom Python script (TLS 1.2, 3 ciphers), hand-crafted raw TLS client
Unknown client	Python 3.14.6 urllib (OpenSSL 3.5.7), deliberately not in the database

Item	Value
Spoofing	hand-crafted client replaying Chrome's JA3 string or JA4_r string
Training data	train_v2.pcap: 9 clients x 3 runs (27 runs), 3,709 packets after filtering
Test data	test_v2.pcap: 9 clients x 5 runs, unknown client x 5, 2 spoof kinds x 5 (60 runs), 6,658 packets; 90 labelled ClientHellos
Reference data	10 FoxIO test captures with FoxIO's expected output; Salesforce and FoxIO published values
Parameters	BPF "tcp port 443"; 2 s gap between runs; label window +/-1 s; 64 KiB stream buffer; 120 s idle timeout

Ground truth: clients run strictly one after another, so each ClientHello to the manifest's SNI inside a run's time window gets that run's label. The training and test captures were recorded separately; the database is built only from the training capture plus published entries. The live demo additionally shows OS-level ground truth (the owning process of each local port).

VII. Evaluation

A. Extractor validation

Table III compares our fingerprints with three independent references. FoxIO's expected values come from FoxIO's own implementation on FoxIO's test captures; Wireshark 4.6.7 computes JA3, JA3S and JA4 itself.

TABLE III. VALIDATION AGAINST INDEPENDENT REFERENCES

Reference	JA3	JA3S	JA4	JA4S
Published examples (Salesforce)	2/2	1/1*		
FoxIO expected output, 10 pcaps			135/136**	76/76
FoxIO published Chrome JA4			identical	
Wireshark 4.6.7, our captures	225/225	202/202	225/225***	n/a

* The published JA3S example hash equals MD5 of the string plus a newline character, an artefact of the publisher's tool; MD5 of the string itself is what the method defines and what we compute. ** For a non-ASCII ALPN value (0xBA 0xAD) the written JA4 specification prints hex characters ("bd", our output) while FoxIO's code prints "99": the specification and its reference code disagree. *** 95 identical, 130 identical after removing the GREASE value that Wireshark keeps in the signature algorithms (Section V-A). Our live Chrome JA4 (t13d1517h2_8daaf6152771_cb7bf5808d99) is exactly the value FoxIO publishes for Chrome.

B. Identification accuracy

Table IV and Fig. 2 report the held-out test capture. Accuracy, recall and F1 are computed over the 75 connections of the 9 known clients; a prediction is correct only if it is a single identification naming the true client. Spoofed connections are reported separately but count against the precision of the label they were mistaken for.

TABLE IV. HELD-OUT IDENTIFICATION (90 CLIENTHELLOS)

Method	Known clients	Macro F1	Unknown rejected	Spoofs accepted
JA4 + JA3	75/75 (100%)	0.967	5/5	10/10
JA3 only	44/75 (58.7%)	0.778	5/5	5/10
JA4 only	75/75 (100%)	0.981	5/5	5/10

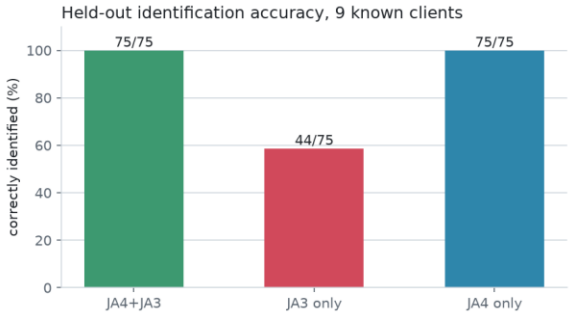


Fig. 2. Held-out accuracy per method (9 known clients).

With JA4 every known client is identified. JA3 alone fails completely for the two Chromium browsers (Chrome 0/12, Edge 0/19) and works for all other clients. Macro F1 stays below 1.0 at 100% accuracy because the 10 spoofed connections were labelled Chrome: Chrome's precision drops to 12/22 = 0.545. The 5 connections from Python 3.14 were all reported as unknown, not guessed, because its newer OpenSSL produces a different fingerprint.

C. Fingerprint stability and randomisation

TABLE V. DISTINCT FINGERPRINTS OVER REPEATED CONNECTIONS

Client	Connections	Distinct JA3	Distinct JA4
Google Chrome	19	19	1
Microsoft Edge	31	31	2
Mozilla Firefox	21	1	1
Each non-browser client	5 to 8	1	1

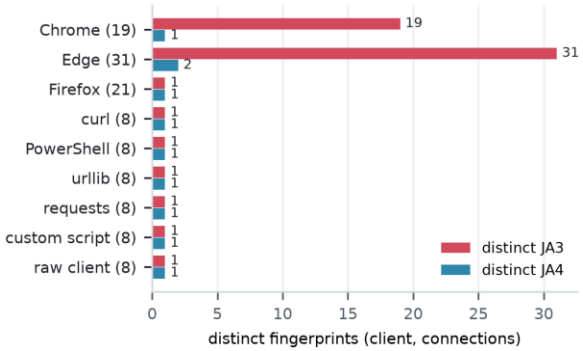


Fig. 3. Distinct JA3 vs JA4 per client over repeated connections (train + test).

Chrome and Edge produced a new JA3 on every connection because they randomise the order of their

TLS extensions, and JA3 hashes extensions in wire order. JA4 sorts extensions before hashing, so Chrome has a single JA4. Edge's two JA4 values differ by exactly one extension, 0x0029 (pre_shared_key), present only when Edge resumes a session. Firefox does not randomise, so even its JA3 is stable. JA4 is not immune to change: any client that changes the set of ciphers or extensions also changes its JA4. Chrome and Edge share BoringSSL: same ciphers and signature algorithms, and Edge's first-connection hello is Chrome's minus a single extension (0xca34). The JA4 cipher hash therefore identifies the TLS library (8daaf6152771 BoringSSL, 2b729b4bf6f3 Windows Schannel for curl and PowerShell, 85036bcba153 OpenSSL 3.0 defaults for urllib and requests), while the other parts separate applications.

D. Spoofing and evasion

The hand-crafted client replayed Chrome's JA3 string (5 connections) and Chrome's JA4_r string (5 connections). Wikipedia's server answered all 10 forged hellos with a ServerHello, and the combined method labelled all 10 as Chrome. Each replay fools only the hash it copies: JA3-only was fooled by the 5 JA3 replays, JA4-only by the 5 JA4_r replays. One weak signal remained: the JA3 replays had status ja3_only (their JA4 was unknown), while all 12 genuine Chrome connections were ja4_only. A JA4_r replay leaves no such trace, and complete tools such as uTLS [9] reproduce full connections. Fingerprints are evidence, not proof of identity.

E. Unknown clients and monitoring

The report command lists observations by status, the most frequent unknown fingerprints (candidates for labelling), conflicts and hits on malware-labelled entries, each with the provenance of the reference entry. The database holds published JA3, JA3S, JA4 and JA4S values for Trickbot, Emotet, IcedID, Sliver, Evilginx, Metasploit and Cobalt Strike with their source. Published attributions are not always consistent: JA3 4d7a28d6... is "Emotet" in the Salesforce README but "Tofsee" in the abuse.ch SSL blacklist [6]. Note that the published IcedID JA4 shares its cipher hash with curl and PowerShell, because all three use Windows Schannel.

F. Server-side fingerprints

TABLE VI. JA4S OF WWW.WIKIPEDIA.ORG PER CLIENT TYPE

JA4S	Reply extensions	Received by
t130200_1301_a56c5b993250c	002b, 0033	TLS 1.3 clients
t120500_c02c_45145cc9ca06c	ff01,0000,000b,0023,0017	TLS 1.2-only script
t130300_1301_0ee26285a86fc	002b, 0033, 0029	Edge resuming

JA3S/JA4S fingerprint the server's reply to a particular client, so one server has several fingerprints (Table VI). More importantly, the JA4S that FoxIO publishes for the Sliver C2 server contains only the two mandatory TLS 1.3 extensions and is sent by Wikipedia and Wikimedia in our captures, and by Microsoft and Google hosts in our first capture; SoftEther VPN's published JA4S is sent by Bing and Office hosts. A JA4S match alone is therefore not evidence of C2, and the tool prints server matches as hints.

G. Performance

TABLE VII. PROCESSING COST ON ONE CORE (2 RUNS)

Measurement	Range
ClientHello parse	50 to 97 microseconds
JA3 + JA4 computation	62 to 103 microseconds
JA3S + JA4S computation	9 to 22 microseconds
Database lookup + identify	4 to 31 microseconds
Offline pipeline incl. Scapy decoding	1,900 to 3,400 packets/s

Timings vary about two-fold with laptop load. Even at the slow end, fingerprinting handles several thousand hellos per second. The pipeline is limited by Scapy's per-packet decoding, which is adequate for an endpoint or a small office link but not for line-rate capture.

VIII. SUMMARY

We built a passive tool that extracts JA3, JA3S, JA4 and JA4S from live or recorded traffic and identifies the client from a curated reference database with explicit evidence statuses. The extractor agrees with FoxIO's published expected output, the published Salesforce examples and Wireshark, apart from three explained cases that trace back to the references themselves. On a held-out capture it identified all 75 connections of 9 real clients with JA4, while JA3 alone identified 44, failing for every Chromium-browser connection because of extension-order randomisation. Unknown clients were reported as unknown.

Limitations. Fingerprints identify a TLS library and configuration, not an application; spoofing is easy (10/10); server fingerprints are generic; fingerprints drift between versions (Python 3.13 and 3.14 already differ), so the database needs maintenance; the evaluation covers one machine, one destination and 90 test connections; QUIC/HTTP3 is not covered (browsers ran with QUIC disabled); and ECH hides the inner hello. **Future work:** QUIC support (JA4 with the q prefix, after removing QUIC Initial packet protection [10]); a kernel capture path (XDP) measured on Linux; combining the TLS fingerprint with HTTP (JA4H) and TCP (JA4T) fingerprints to make spoofing harder; and evaluation on more hosts, operating systems and destinations.

IX. REPRODUCIBILITY

GitHub:
https://github.com/SailajaMudu/TLS_Fingerprinting

The repository contains the training and test captures with their manifests, the reference database, the published references with source URLs, FoxIO's test captures with their expected output, and all scripts. On Windows with Python 3.13 (whose OpenSSL matches the database):

```
py -3.13 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt matplotlib requests
# tests incl. published vectors and FoxIO pcaps
python -m pytest -q tests
# regenerate all results (~1 min):
#   results/*.csv, chart *.png, SUMMARY.md
python experiments\run_experiments.py
# demonstration (live step needs Npcap)
.\demo\run_demo.ps1 -Interface "Wi-Fi 3"
```

New captures are recorded by running `python -m tlsfp live -i <iface> -w raw.pcap` in one terminal and `python scripts/gen_traffic.py --repeat 5 -o manifest.json` in another, then removing unrelated traffic with `scripts/filter_pcap.py`. The README lists the exact commands, including the spoofing and unknown-client options.

REFERENCES

[References do not count toward the 10-page limit.]

[1] J. Althouse, J. Atkinson, and J. Atkins, "TLS fingerprinting with JA3 and JA3S," Salesforce Engineering Blog, Jan. 2019. [Online]. Available: https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967/

[2] Salesforce, "JA3 - A method for profiling SSL/TLS clients," GitHub repository (archived 2025). [Online]. Available: https://github.com/salesforce/ja3

[3] FoxIO, "JA4+ network fingerprinting" and "JA4 technical details," GitHub repository. [Online]. Available: https://github.com/FoxIO-LLC/ja4

[4] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," RFC 8446, IETF, Aug. 2018.

[5] M. Husák, M. Čermák, T. Jirsík, and P. Čeleda, "Network-based HTTPS client identification using SSL/TLS fingerprinting," in Proc. 10th Int. Conf. Availability, Reliability and Security (ARES), Toulouse, France, 2015, pp. 389-396.

[6] abuse.ch, "SSL Blacklist: JA3 fingerprints," last updated Aug. 2021. [Online]. Available: https://sslbl.abuse.ch/blacklist/ja3_fingerprints.csv

[7] Chrome Platform Status, "TLS ClientHello extension permutation," enabled by default in Chrome 110, 2023. [Online]. Available: https://chromestatus.com/feature/5124606246518784

[8] D. Benjamin, "Applying Generate Random Extensions And Sustain Extensibility (GREASE) to TLS extensibility," RFC 8701, IETF, Jan. 2020.

[9] Refraction Networking, "uTLS: fork of the Go standard TLS library providing low-level access to the ClientHello for mimicry purposes," GitHub repository. [Online]. Available: https://github.com/refraction-networking/utls

[10] M. Thomson and S. Turner, "Using TLS to secure QUIC," RFC 9001, IETF, May 2021.

[11] Wireshark Foundation, "Wireshark 4.6 / TShark manual," 2025. [Online]. Available: https://www.wireshark.org/docs/

[12] P. Biondi and the Scapy community, "Scapy: packet manipulation library," 2025. [Online]. Available: https://scapy.net

Appendix A: Team Work Distribution and Member Rating

A.1 Work Distribution

- Ratings are agreed by the whole team and are unique (no ties); scale 1 to 10 (10 = highest contribution).

#	Member Name	Roll No.	Tasks Done	Rating (1-10, unique)
1	Vadithya Harsha Vardhan Nayak	23110349	Capture and protocol core: capture, TCP reassembly, parser, JA3/JA3S/JA4/JA4S, XDP source, tests, Wireshark/FoxIO/Salesforce validation and documented the Report	10
2	Sailaja Mudu	23110288	Reference database and identification: SQLite database, identification statuses and tests, published fingerprints with sources, Redis, CLI and monitoring report, security analysis and documented the Report	9
3	Banoth Arvindh	23110058	Experiments, evaluation and demo: traffic generator, spoofing client, captures, experiment script with results and charts, demo script, TRex profile, README and documented the Report	8

Appendix B: AI Usage Details

B.1 Prompts

The prompts and responses for the uses below are stored as Markdown in the repository folder `AI Prompts/`. All AI-generated code and text was reviewed, and every reported number was produced by running the committed scripts; the AI was also used to check the report's claims against the result files.

#	AI Tool	Purpose
1	Claude Code (Claude Opus 5.5)	Code generation and implementation assistance throughout development
2	ChatGPT (GPT-6)	Concept explanations, debugging discussions, understanding implementation behaviour, experiment interpretation and report preparation